

Plant Phenotyping via Geometric Supervised Deep Learning on 3D Point Clouds

Angel A. Barrera-Gomez, Inhwan Jung, and Luke Hussung

College of Arts and Sciences, Department of Mathematics & Statistics

East Tennessee State University in collaboration with Oak Ridge National Laboratory

December 5, 2025

Abstract

Abstract: High-throughput phenotyping is essential for optimizing bioenergy feedstocks to meet global renewable energy demands. While traditional Computer Vision (CV) techniques have revolutionized 2D image analysis for agriculture, they often lack the volumetric precision required for complex woody plants. This study presents a 3D Deep Learning pipeline developed for the Advanced Plant Phenotyping Laboratory (APPL) at Oak Ridge National Laboratory. We propose a Dynamic Edge Convolutional Neural Network (DECNN) to segment high-resolution LiDAR point clouds of *Populus* (York Poplar) into biologically meaningful classes: Stem, Leaf, and Support Stake. By integrating explicit geometric feature engineering (Linearity, Planarity, Sphericity) with dynamic graph learning, we achieved a Global Mean Intersection over Union (IoU) of 65.58% and an Overall Accuracy of 88.99%. Notably, the model achieved a Stem Recall of 82.27%, successfully reconstructing the primary growth axis of the plants despite severe class imbalance. This approach effectively automates the separation of biological structures from abiotic support infrastructure, overcoming significant geometric ambiguity in forestry phenotyping.

Keywords: 3D Phenotyping; LiDAR; Bioenergy; Dynamic Graph CNN; Semantic Segmentation.

1 Introduction

Developing sustainable, high-yield bioenergy feedstocks is a critical component of future energy security. To optimize species such as *Populus* (Poplar) for biofuel production, researchers must identify genotypes that exhibit superior growth characteristics and biomass accumulation suitable for conversion into biofuels. However, this requires the accurate assessment of plant architecture at a scale that manual measurement cannot achieve. Computer Vision (CV) combined with Deep Learning has emerged as a reliable method to characterize and quantify plant traits. Since 2012, Convolutional Neural Networks (CNNs) have dominated CV tasks, demonstrating superior performance over traditional machine learning methods [3]. While these technologies have been successfully applied to agricultural grain crops [1], their application to complex, woody bioenergy plants presents unique challenges regarding structure and occlusion. While 2D imaging has advanced rapidly, it suffers from inherent limitations in measuring volumetric traits such as total woody biomass and complex branching angles due to occlusion. The Advanced Plant Phenotyping Laboratory (APPL) at Oak Ridge National Laboratory utilizes Light Detection and Ranging (LiDAR) to capture 3D point clouds of York Poplar trees. This 3D data offers a pathway to extract precise morphological phenotypes essential for bioenergy research.

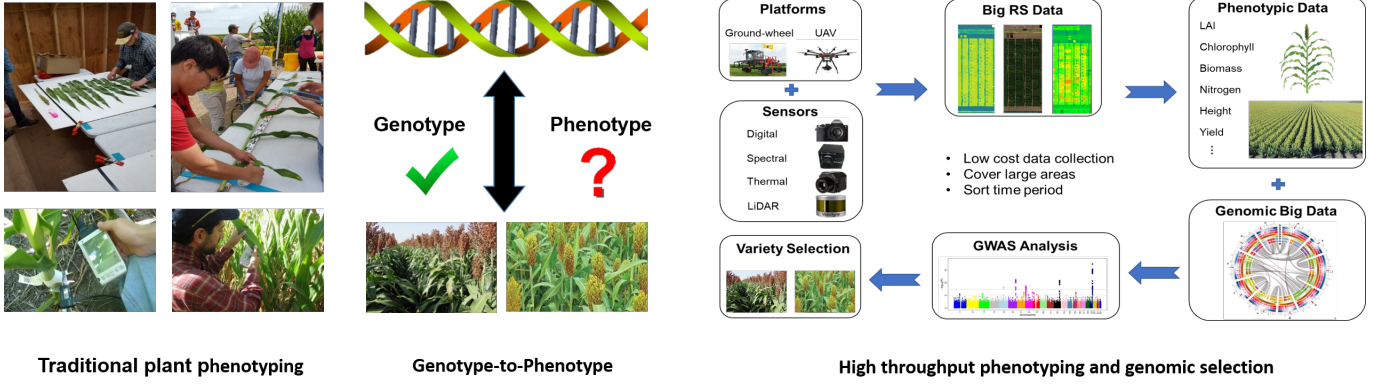


Figure 1: Comparison of traditional manual phenotyping (left) versus modern high-throughput phenotyping pipelines (right) that utilize sensors such as LiDAR and spectral cameras to bridge the genotype-to-phenotype gap [4].

2 Preliminaries

This section outlines the foundational concepts required to understand the proposed method to solve plant phenotyping using a Supervised Deep Learning approach, bridging the gap from traditional grid-based processing to geometric deep learning on unstructured point clouds.

2.1 Traditional Convolution Layer

Generally, convolution refers to a filtering operation in which a filter (kernel) is applied to an input to generate new feature maps. Standard convolution operations perform multiplication and summation on the input with a filter sliding along spatial dimensions. For example, given a 2D image with height H , width W , and channels C , a kernel of size $3 \times 3 \times C'$ is applied to produce an output feature map.

$$y_{i,j} = \sum_{u,v} w_{u,v} \cdot x_{i+u,j+v} + b \quad (1)$$

Traditional convolution relies heavily on ordered, structured data (grids). This limits its direct application to 3D point clouds, which are unordered sets of continuous coordinates without a fixed spatial grid structure.

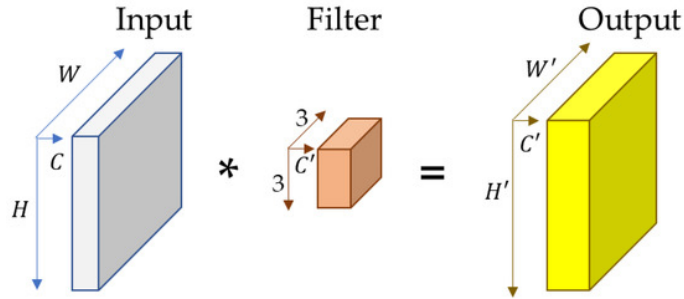


Figure 2: Traditional convolution operation with kernel size of $3 \times 3 \times C'$, where C' indicates the output dimension of the feature maps [2].

2.2 Graph Convolution and Edge Convolution

To process unstructured data such as point clouds, we represent the data as a graph $G = (V, E)$, where V represents the nodes (points) and E represents the edges connecting neighbors.

Unlike traditional methods that operate on fixed grids, **Edge Convolution** operates on local neighborhoods to extract geometric features. For a point x_i and its neighbor x_j , the edge feature is defined as $e_{ij} = h_{\Theta}(x_i, x_j - x_i)$, where h_{Θ} is a nonlinear function (typically a Multi-Layer Perceptron) with learnable parameters Θ . By including both the global coordinates (x_i) and the local vector ($x_j - x_i$), the network captures both absolute position and local geometric shape.

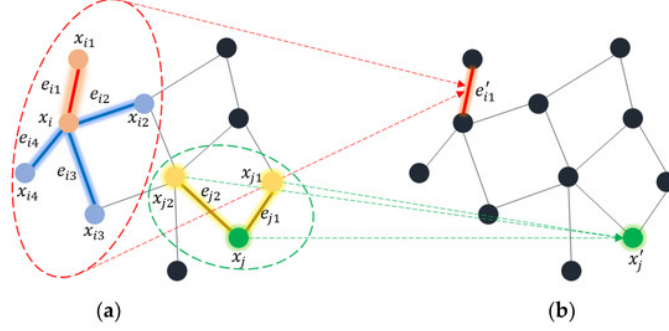


Figure 3: Graph convolution operations: green and red dotted circles represent (a) node and edge convolution operations neighbors on the left side and (b) convolution outputs on the right side [2].

2.3 Application to 3D Plant Phenotyping

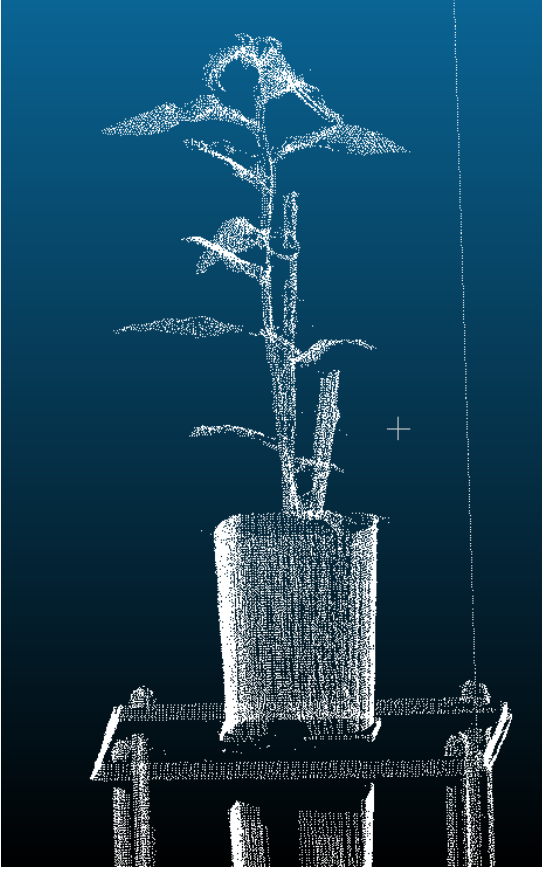
While traditional CNNs (Figure 2) excel at processing dense, grid-structured data like RGB images, they are ill-suited for raw LiDAR data. Point clouds are inherently sparse and unordered; distinct plant organs are defined by their geometric structure rather than their position on a fixed grid. Converting these sparse clouds into dense 3D voxels to satisfy standard CNN requirements would result in significant computational inefficiency, as the vast majority of the volume is empty space (air).

In contrast, the Graph/Edge Convolution approach (Figure 3) aligns perfectly with the morphological analysis required for phenotyping. By treating each LiDAR point as a node in a graph, the model can dynamically aggregate information from its nearest physical neighbors. This allows the network to learn geometric primitives. For instance, recognizing that a vertical linear arrangement of neighbors indicates a *Stem*, while a planar arrangement indicates a *Leaf*, regardless of where the plant is positioned in the scanner’s field of view. The dynamic edge operation (e_{ij}) explicitly captures these local shape descriptors, making it the superior architecture for isolating complex biological structures from abiotic noise.

3 Materials and Methods

3.1 Data Acquisition

The dataset consists of high-resolution LiDAR scans of York Poplar trees collected at APPL between March 27 and June 26, 2024. The data captures the complete developmental trajectory of the plants during their early growth phase.



(a) 3D Point Cloud of York Poplar.



(b) RGB photo and laser scan York poplar tree.

Figure 4: Comparison of raw 3D data (a) and the broader phenotyping context (b)

3.2 Manual Annotation Protocol

To generate high-fidelity ground truth data for supervised learning, we established a rigorous manual annotation protocol utilizing the CloudCompare software environment. Our intention was to set a step-by-step guide that can be replicated by other users, in order to set a true value in which we start our data pipeline. Here, we make sure the pre-processing account for precision, and fidelity. Especially for the noise and outliers naturally seen in this type of data. We fully annotated 30 distinct point clouds, ensuring a diverse representation of plant morphologies. The annotation workflow employed a hierarchical subtraction strategy to ensure geometric consistency:

1. **Segmentation Strategy:** Annotators utilized the polygonal segmentation tool to isolate regions of interest. The protocol mandated a "coarse-to-fine" approach, where large abiotic artifacts (Pot, Background) were segmented out first to reduce scene complexity. The remaining biological structure was then iteratively partitioned using "Segment In" and "Segment Out" operations to isolate the stem and individual leaves.
2. **Boundary Definition:** Precise boundary rules were enforced to resolve geometric ambiguity. Specifically, the boundary between *Stem* and *Leaf* was defined at the natural joint of the petiole. In instances of point overlap or occlusion, priority was given to preserving the continuity of the stem segment.
3. **Completeness Constraint:** To prevent unlabeled artifacts that could confuse the loss function, a strict completeness rule was applied: every single point in the raw cloud was required to be assigned to

a specific class. Unassigned points were investigated and manually merged into the nearest appropriate class.

4. **Label Encoding:** Once segmented, the sub-clouds were merged by category and assigned a scalar classification field (Scalar Field index) for model ingestion, exported as a .txt file, and deposited in a repository that is separated from the model's data folder. The reason being, we wanted to make sure we could preserve the integrity of the files; especially if we ever need to go back and modify any point cloud without altering the data we have fed the model before.

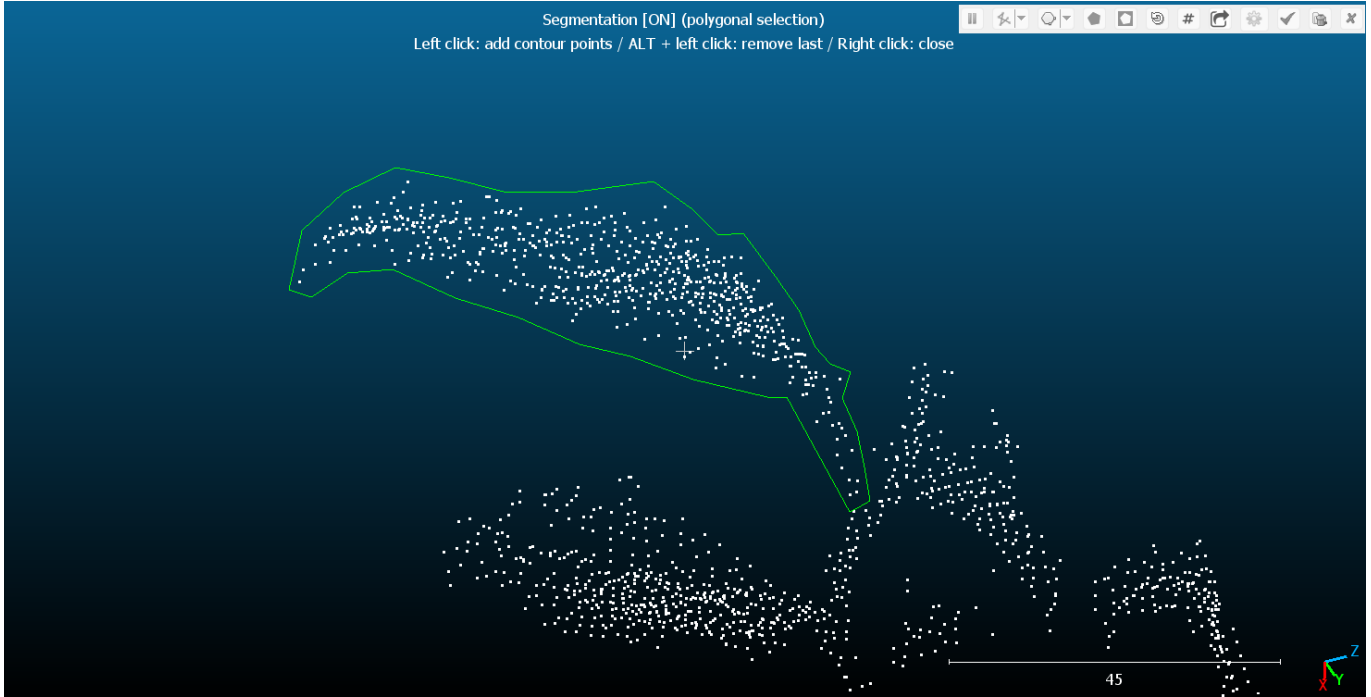
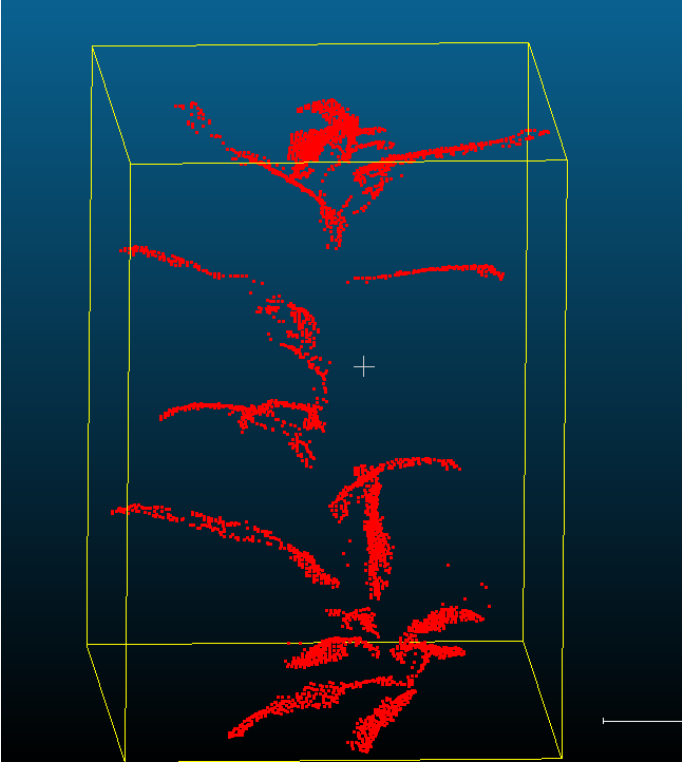


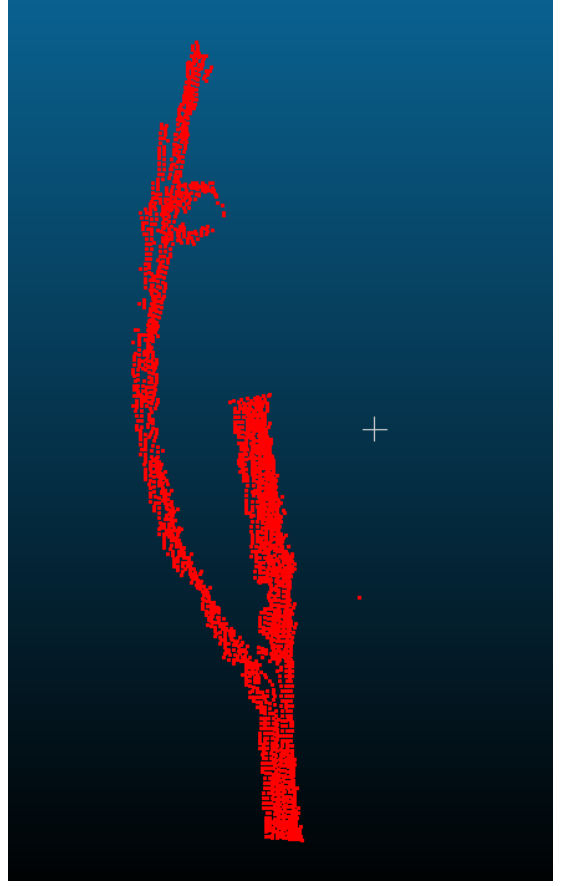
Figure 5: Manual annotation workflow in CloudCompare.

The resulting dataset classifies every point into one of four distinct categories:

- **Stem (Class 0):** The central woody axis of the plant.
- **Leaf (Class 1):** All photosynthetic tissue and petioles.
- **Stake (Class 2):** The synthetic support structure used to guide vertical growth.
- **Background (Class 3):** Non-biological elements including the pot, tray, and sensor noise (filtered during preprocessing).



(a) Isolated Leaf Class



(b) Isolated Stem Class

Figure 6: Ground truth generation. Note the precise separation of the stem (a) from the petioles in the leaf class (b). Some noise is still included.

4 MLOps and Infrastructure Engineering

4.1 Transition to Modular Architecture

Initial exploratory data analysis (EDA) and prototype modeling were conducted using interactive Jupyter Notebooks. While effective for rapid visualization, this monolithic approach proved unsuitable for rigorous hyperparameter tuning, version control, and reproducibility. Consequently, the codebase was refactored into a basic production-level MLOps framework. Core functionalities were decoupled into distinct modules located in the `src/` directory, separating the model definition from the execution logic. This modularity facilitates unit testing and allows for the seamless integration of new architectures without disrupting the data pipeline.

4.2 Project Directory Structure

The project infrastructure is designed to ensure a clean separation between raw data ingestion, model definition, and training orchestration (Figure 7). This structure enforces data integrity and reproducibility, key tenets of robust ML engineering.

- **Data Management (data/):** The dataset is split into `train`, `val`, and `test` directories. To preserve

data integrity, annotated ASCII point clouds are deposited in the **raw/** subfolders. The training script (**train.py**) processes these into tensor files saved in **processed/**, ensuring the original labels are never modified.

- **Source Code (src/):** Core logic is decoupled from execution scripts. This includes **model.py** (architecture definition), **dataset.py** (ETL pipeline), and **inference.py** (visualizer).
- **Execution & Validation:** The root directory contains the main drivers: **train.py** for the training loop and **evaluation.py** for metric auditing. A dedicated **validations/** folder houses scripts for EDA and integrity checks (e.g., detecting NaNs or rogue labels) before training begins.

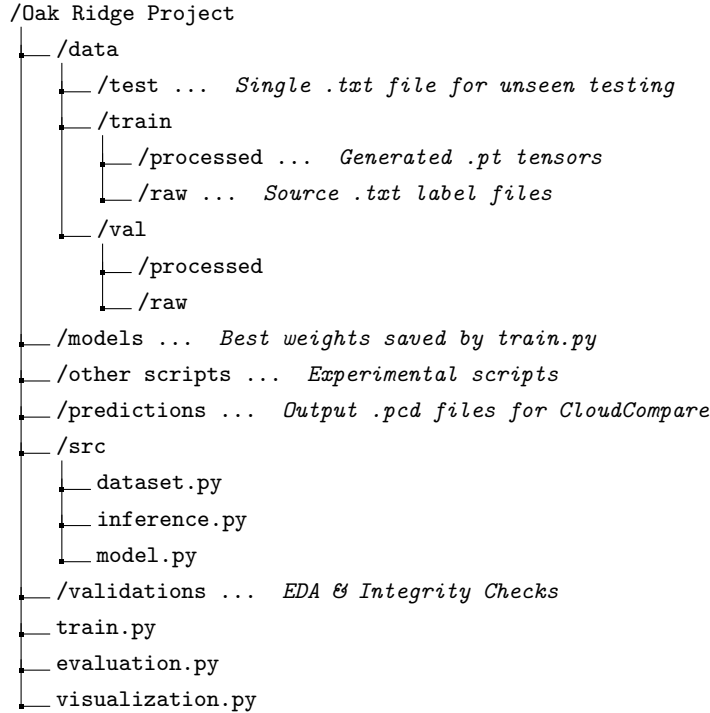


Figure 7: Project Directory Map. The architecture strictly separates raw data ingestion (handled in **data/**) from model logic (**src/**) and execution drivers (root), ensuring reproducible and corruption-free training cycles.

4.3 Future Engineering: Towards Scalable Deployment

The modularization of the codebase opens the door to the fundamental need for containerization and distributed computing. Our current work has decoupled the model logic from the execution environment, satisfying the prerequisites for **Docker** integration.

Future engineering efforts could focus on encapsulating the training and inference pipelines into immutable containers. This could ensure cross-platform reproducibility, allowing the model to be deployed on diverse hardware at ORNL without dependency conflicts. Furthermore, this containerization paves the way for orchestration via **Kubernetes**, enabling the horizontal scaling of the inference pipeline to process high-throughput datasets across distributed computing clusters.

5 Feature Engineering

Raw XYZ coordinates are often insufficient for deep learning models to distinguish between flat leaves and cylindrical stems. To augment the input space, we computed local geometric descriptors for every point p_i using a k -nearest neighbor approach ($k = 20$) within a radius $r = 0.05m$.

For each point p_i , we construct a local covariance matrix $\mathcal{C}$ based on its neighbors:

$$\mathcal{C} = \frac{1}{k} \sum_{j=1}^k (p_j - \bar{p})(p_j - \bar{p})^T \quad (2)$$

where $\bar{p}$ is the centroid of the neighborhood. The eigenvalues of $\mathcal{C}$ are computed and sorted such that $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$.

These eigenvalues describe the spatial distribution of the neighborhood, allowing us to derive the following shape descriptors:

- **Linearity (L_λ):** Measures how elongated the neighborhood is (critical for Stem/Stake detection).

$$L_\lambda = \frac{\lambda_1 - \lambda_2}{\lambda_1} \quad (3)$$

- **Planarity (P_λ):** Measures the flatness of the surface (critical for Leaf detection).

$$P_\lambda = \frac{\lambda_2 - \lambda_3}{\lambda_1} \quad (4)$$

- **Sphericity (S_λ):** Measures how isotropic (scattered) the points are (useful for complex junctions).

$$S_\lambda = \frac{\lambda_3}{\lambda_1} \quad (5)$$

- **Relative Height (H_{rel}):** Normalizes the Z-coordinate to provide vertical context relative to the plant’s total height.

$$H_{rel} = \frac{z_i - z_{min}}{z_{max} - z_{min}} \quad (6)$$

This engineering process transforms the raw input into a robust 7-dimensional vector $[n_x, n_y, n_z, L_\lambda, P_\lambda, S_\lambda, H_{rel}]$ for every point fed into the network.

6 Network Architecture: Dynamic Edge CNN

We implemented a Dynamic Edge CNN (DECNN). This architecture constructs a local graph for each point and updates the graph connections dynamically at each layer.

- **Graph Construction:** A radius-based graph ($r = 0.05m$) connects points that are spatially close, allowing the model to “see” local geometry.
- **Edge Convolution:** The network computes features based on the relationship between a point and its neighbors, effectively learning edge features (like the curve of a stem) rather than just point features.

- **Loss Function:** To address the severe class imbalance (where leaves account for $> 75\%$ of points), we utilized a composite loss function of Weighted Cross Entropy and Dice Loss, with class weights of $[5.0, 1.0, 5.0]$ for Stem, Leaf, and Stake, respectively.

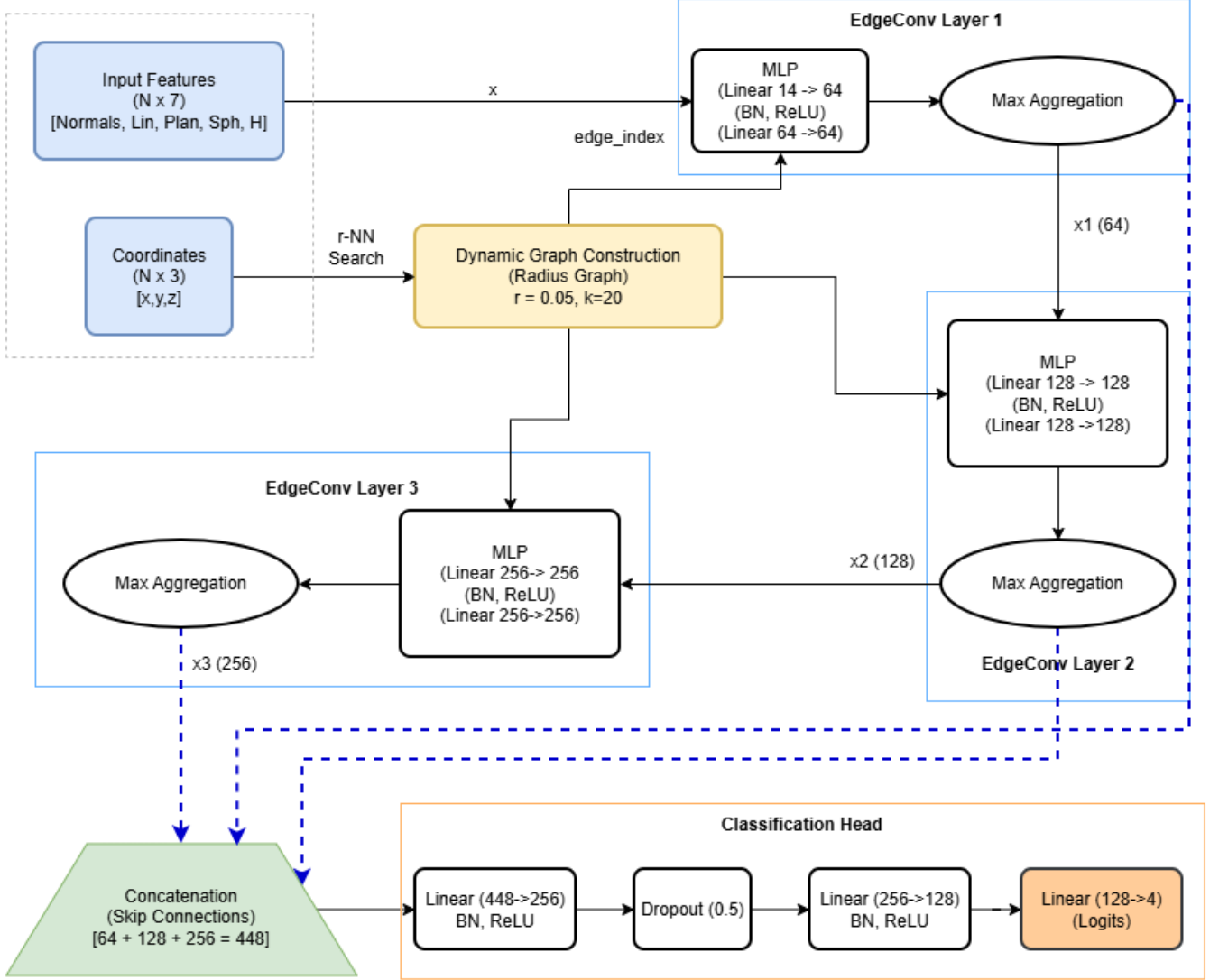


Figure 8: Architecture of the Dynamic Edge CNN. The model takes 7 geometric features as input, processes them through three dynamic edge convolution blocks, and aggregates features to classify each point as Stem, Leaf, Stake, or Background.

7 Experimental Results

7.1 Quantitative Performance

Performance was evaluated using the Intersection over Union (IoU) metric on a hold-out validation set. Table 1 summarizes the performance metrics.

Class	IoU (%)	Recall (%)	Precision (%)	Support
Stem	53.87	82.27	60.94	8,330
Leaf	92.17	93.64	98.33	45,162
Stake	50.70	62.94	72.27	5,899
Global Mean	65.58	-	-	-
Overall Accuracy	88.99	-	-	-

Table 1: Performance metrics on the validation dataset. The model demonstrates high recall for the critical Stem class.

The model achieved a **Sample-Averaged Mean IoU of 67.21%** with a 95% confidence interval of [61.86% – 71.62%]. The results indicate robust segmentation of the dominant Leaf class (IoU > 92%) and successful retrieval of the Stem (Recall > 82%).

7.2 Visual Analysis

Qualitative inspection reveals that the model successfully reconstructs the primary plant structure. However, a trade-off was observed regarding the Stem class due to resolution constraints.

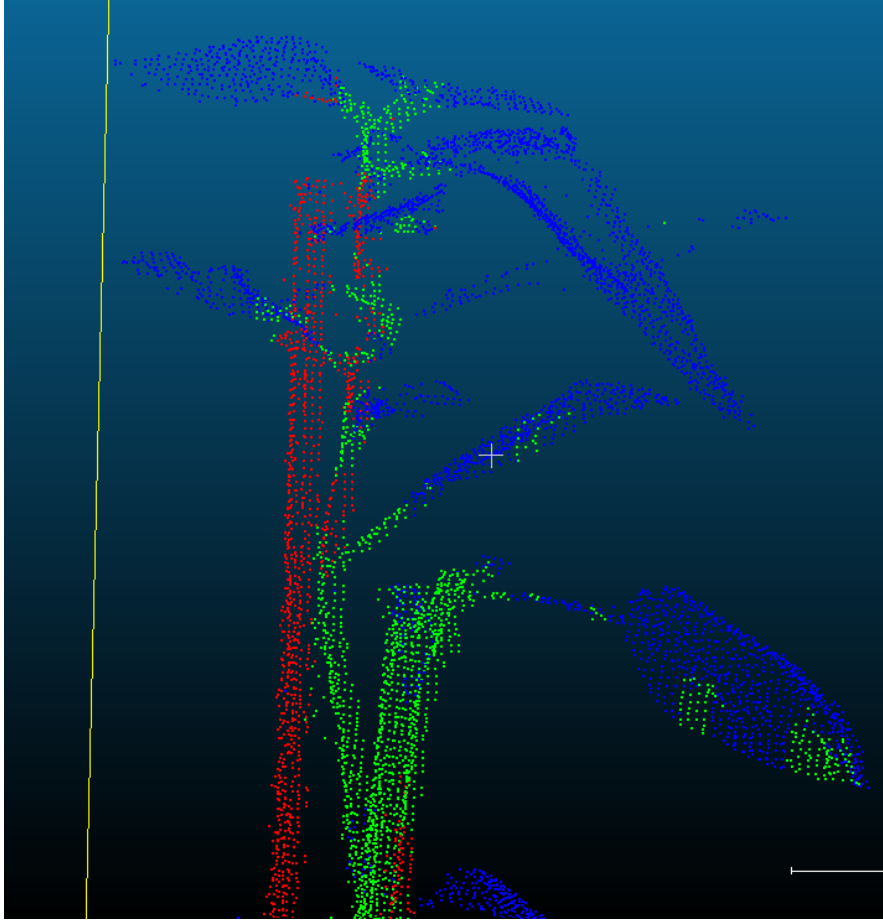


Figure 9: Visual inference result on a test sample. The DECNN model successfully segments the point cloud into semantic classes: **Stem (Green)**, **Leaf (Blue)**, and **Stake (Red)**. The visualization highlights the model’s ability to reconstruct the vertical stem structure even in regions of high occlusion.

8 Results Discussion

8.1 The Recall-Precision Trade-off

A key challenge in Deep Learning is class imbalance. In our case, the Stem is a minority class. By applying a heavy class weight (5.0), we successfully forced the model to prioritize Stem detection, achieving an 82.27% Recall. However, this resulted in a “thickening” artifact, where the predicted stem diameter exceeds the ground truth, bleeding into the petioles. This lowers the Precision (60.94%) and IoU. For phenotyping purposes, high Recall is preferred, as post-processing skeletonization can refine a thick stem, whereas a missing stem (low Recall) cannot be recovered.

8.2 Stem vs. Stake Ambiguity

Distinguishing the Stem from the Stake remains the most significant challenge. Both objects are vertical cylinders with similar eigenvalues for Linearity and Planarity. The model achieved a Stake IoU of 50.70%, which is promising given the lack of color data. In many cases, the model correctly identified the stake based on its perfect straightness compared to the organic curvature of the living stem.

9 Challenges and Limitations

9.1 Geometric Ambiguity without Spectral Data

The most persistent challenge encountered was the discrimination between the biological *Stem* and the synthetic *Stake*. Geometrically, both structures present as thin, vertical cylinders with comparable eigenvalues for linearity (L_λ). In the absence of RGB spectral data, which would easily distinguish the brown wood from the green plant tissue, the model relies exclusively on subtle geometric cues, such as the “perfect straightness” of the stake versus the organic curvature of the stem. This resulted in a Stake IoU of only 50.70%, with misclassification frequently occurring at the specific points where the plant is tied to the support structure.

9.2 Hardware Constraints and Voxel Resolution

To accommodate the memory constraints of the available hardware (8GB VRAM), the input point clouds were downsampled to a voxel size of $2cm$ ($0.02m$). While necessary for training stability, this resolution is coarse relative to the physical diameter of the young stems ($\approx 0.5 - 1.0cm$). This introduces a “Partial Volume Effect,” where a single voxel may contain points from both a stem and a nearby petiole, leading to inevitable classification bleed at the organ boundaries.

9.3 The Dilation Artifact

To counteract the severe class imbalance, aggressive weights (5.0) were assigned to the Stem class during training. While this successfully drove Stem Recall to 82.27%, ensuring the stem is rarely missed, it induced a “dilation artifact” where the model over-predicts the stem volume to ensure continuity. This

effectively "thickens" the digital representation of the stem, lowering Precision to 60.94%. While this artifact complicates diameter estimation, prioritizing Recall was a strategic decision to ensure accurate height measurement, which relies on unbroken vertical continuity.

10 Conclusion and Future Work

This study demonstrates that Dynamic Graph CNNs can effectively automate the segmentation of 3D plant phenotypes, overcoming the limitations of traditional manual measurement. Achieving a Global Mean IoU of 65.58% and successfully automated the isolation of woody stems. But the biggest takeaway, is that the model successfully identified and discriminated the different organs of the plant. Especially the leaves with an impressive 93.64% recall, and most importantly a 92.17% IoU. This is essentially like having a true value and the prediction being almost identical to that. This opens up the door to a deeper effort in analyzing the leaves. Categorizing, and identifying each individual leaf can help us understand the very biological characteristics of them, and with this, obtain significant indicators of a plant health state. Most plants can show signs of sickness if you look close to their leaves.

Future work may focus on integrating RGB data to resolve the Stem/Stake ambiguity using spectral features. Additionally, we aim to explore hierarchical models such as PointNet++ and instance segmentation techniques to separate individual leaves, further advancing the capability of high-throughput phenotyping systems.

References

- [1] Wang, Y.-H., & Su, W.-H. (2022). Convolutional Neural Networks in Computer Vision for Grain Crop Phenotyping: A Review. *Agronomy*, 12(11), 2659. <https://doi.org/10.3390/agronomy12112659>
- [2] Tasnim, N., & Baek, J.-H. (2023). Dynamic Edge Convolutional Neural Network for Skeleton-Based Human Action Recognition. *Sensors*, 23(2), 778. <https://doi.org/10.3390/s23020778>
- [3] O'Shea, K., & Nash, R. (2015). An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*. <https://arxiv.org/abs/1511.08458>
- [4] University of Wisconsin-Madison. (n.d.). *High throughput plant phenotyping*. Digital Agriculture Lab. Retrieved December 12, 2025, from <https://digitalag.bse.wisc.edu/high-throughput-plant-phenotyping/>